COMP4135 SEM Weekly Report 3

Group 2 (Tactical Cards)

Team members

Guangjun HU (20511787)
Hongshuo HU (20808603)
Cheng YE (20513323)
Zehan CHEN (20807750)

School of Computer Science
University of Nottingham

Submission Date: 17/04/2026

Contents

1 Process Management **3**
- 1.1 Management Objective . . . 3
- 1.2 Brief Alignment . . . 4
- 1.3 Explicit Success Criteria . . . 4
- 1.4 Process Model Choice . . . 5
- 1.5 Milestones and Owners . . . 7
- 1.6 Workflow and Repository Control . . . 8

2 Test Strategy **10**
- 2.1 Verification and Validation Roles . . . 10
- 2.2 TDD Use . . . 10
- 2.3 Strategy Matrix . . . 12
- 2.4 Environment and Tooling . . . 13
- 2.5 Scope and Code-Change Guardrails . . . 14
- 2.6 Coverage Strategy . . . 14

3 Test Plan **15**
- 3.1 Requirements Coverage Matrix . . . 15
- 3.2 Module and Class Coverage Summary . . . 16
- 3.3 Final Suite Inventory . . . 17

4 Validation Results **18**
- 4.1 Execution Summary . . . 18
- 4.2 Coverage and Traceability Results . . . 19
- 4.3 Defect Summary . . . 21

5 Summary and Reflections **23**
- 5.1 Outcome . . . 23
- 5.2 Milestone Closure . . . 24
- 5.3 Optional Follow-up Backlog . . . 25
- 5.4 Reflection . . . 25

Reference **26**

A Supporting Evidence Index **27**
- A.1 Testing and Validation Meeting . . . 29
  - A.1.1 Attendance . . . 29
  - A.1.2 Agenda . . . 29

A.1.3 Progress Since Last Meeting 29
A.1.4 Key Discussion Points 29
A.1.5 Decisions Made 30
A.1.6 Action Items 30
A.1.7 Issue Closure Status 30
A.1.8 Next Meeting 30

B Representative Test Case Inventory **31**

Chapter 1

Process Management

This work was executed as a short testing sprint and evidence-freeze cycle rather than a feature-expansion week. The report therefore keeps the required template structure, but frames the work as a weekly sprint with fixed scope, explicit owners, and evidence gates that were checked again on 2026-04-17 [3, 4, 2].

The working plan and milestone ownership are recorded in ‘wr3-execution-plan-2026-04-17.md‘ and ‘wr3-milestones-and-owners-2026-04-17.md‘. Those process records defined the sprint scope, the owner split, and the acceptance gates used to judge when authored work could be promoted into verified evidence.

1.1 Management Objective

The management objective was to convert the existing single-battle prototype into a verifiable, traceable, and reportable weekly checkpoint without reopening gameplay scope. That objective produced three management rules:

- keep gameplay scope fixed at the current single-battle vertical slice rather than adding unrelated features;
- treat automated tests, execution logs, and defect records as first-class deliverables;
- separate *implemented* work from *verified* work so the report does not claim evidence that has not yet been rerun.

1.2 Brief Alignment

Assessor-facing item	Where the report proves it	How the current draft makes it explicit
Testing strategies	Chapter 2 verification/validation table and strategy matrix	The layered approach is stated directly instead of being left implicit in the suite list.
Test types	Chapter 2 strategy matrix plus Chapter 3 suite inventory	EditMode, scene-coupled EditMode, PlayMode smoke, and manual follow-up are all visible to the marker.
Success criteria	Chapter 3 success-criteria table and Chapter 4 execution summary	Pass conditions are quantified explicitly rather than inferred from scattered status language.
Issues tracked and addressed	Chapter 4 defect tables plus the synced defect log	Each defect now has a status, mitigation, and prevention note, including the two inherited WR2 UI issues.
Planning and team roles	Chapter 1 milestone and owner tables	Named owners, deliverables, and exit gates show how the sprint work was allocated.
Repository usage	Chapter 1 workflow figure and repository-control table	Branch choice, rerun rules, and evidence controls are presented as process constraints rather than hidden assumptions.
Attendance and reflection	Appendix meeting minutes and Chapter 5 sprint appraisal	Attendance is recorded historically and the end of the report now contains an explicit weekly sprint appraisal.

Table 1.1: Direct mapping between the brief and the evidence made visible in this report.

1.3 Explicit Success Criteria

The brief expects the report to state *success criteria* directly rather than leaving them to be inferred from later status language. For this reason, the sprint used the following assessor-facing pass conditions as explicit completion gates.

Success criterion	Final status	Visible evidence in this report
All scoped automated tests pass in the final rerun set.	Satisfied	Chapter 4 reports ‘57/57‘ EditMode passes, ‘1/1‘ PlayMode smoke pass, and ‘58/58‘ total rerun-backed execution evidence.
All ‘10‘ scoped requirements have rerun-backed evidence.	Satisfied	Chapter 3 requirements coverage matrix maps every requirement to named verified suites.
At least one end-to-end PlayMode smoke path passes.	Satisfied	The ‘BattleFlowPlayModeTests‘ smoke checkpoint is listed in Chapters 3 and 4 as passed and archived.
All tracked bugs are classified, and any still-open item would have to be marked blocker or non-blocker explicitly.	Satisfied	Chapter 4 defect tables classify inherited WR2 and current-cycle issues by severity and status; no scoped blocker remains open at freeze time.
Coverage evidence is archived and report-ready.	Satisfied	Chapter 4 reports the archived export, including ‘74.2%‘ line coverage and the archived ‘coverage/2026-04-17/‘ evidence pack.

Table 1.2: Explicit success criteria used as the sprint exit gate.

These criteria were used as the practical sprint exit gate: a milestone could be treated as complete only when its output was both present in the repository and visible again as verified or archived evidence in the final 2026-04-17 execution set.

1.4 Process Model Choice

A Waterfall-style approach was considered first because the report has a fixed template, a fixed submission date, and a document-heavy deliverable set. Those characteristics suit up-front planning, milestone locking, and clear owner allocation. However, a pure Waterfall model would have been too rigid for the actual sprint pattern of this cycle: once testing expanded, the team had to iterate on Unity-specific setup issues, repair test discovery gaps, refine PlayMode smoke coverage, and stabilise the coverage export before the final figures were trustworthy.

For that reason the team used a *lightweight Agile* process instead. The report still kept explicit milestones and owner assignments, but the execution model was iterative: small batches of tests were added, rerun, fixed, and then folded back into the same ‘test‘ integration branch. This preserved planning discipline without pretending that the validation work could be completed in one linear pass.

Approach	Why it was considered	Why the final process differed
Waterfall	The report has predefined chapters, known submission constraints, and a strong documentation component.	Too rigid for toolchain debugging, smoke-test repair, and coverage-export integration that only became clear during execution.
Lightweight Agile	Supports short validation loops, defect-driven reprioritisation, and repeated reruns after each fix.	Chosen because it matched how Unity testing and evidence collection actually progressed in this project.

Table 1.3: Why the weekly testing sprint used lightweight Agile rather than a pure Waterfall process.

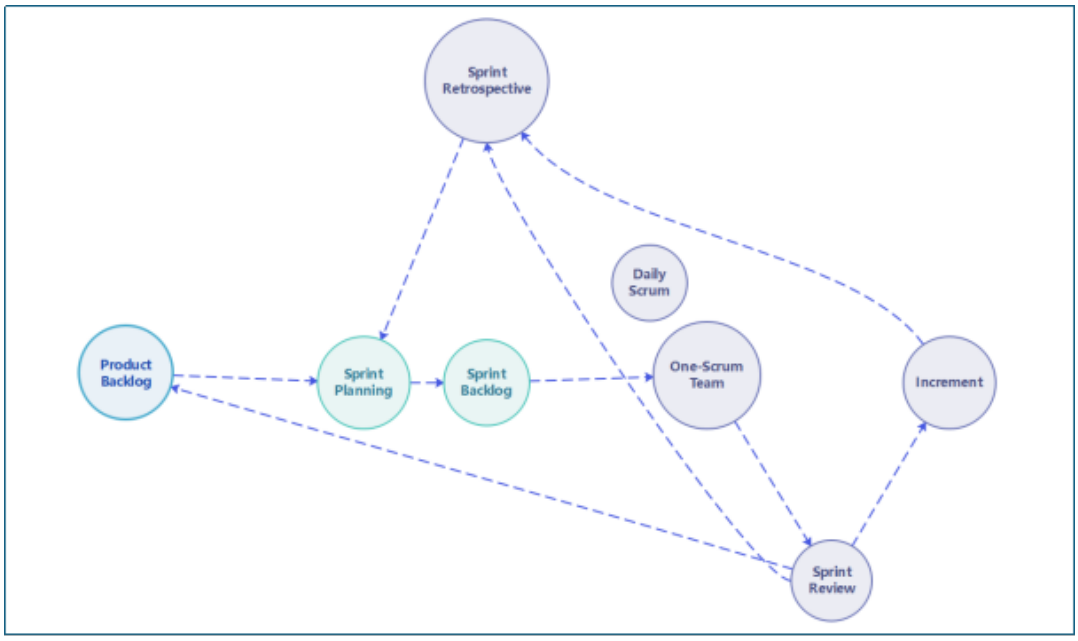

Figure 1.1: Lightweight Agile collaboration model used for planning, backlog refinement, review, and iteration during the testing cycle. This figure shows the team-level process model rather than the evidence workflow shown later in Figure 1.3.

1.5 Milestones and Owners

Milestone	Primary owner	Deliverables	Exit gate
Scaffold and planning setup	Guangjun HU	Report source tree, requirements list, module list, and test inventory	The source tree exists and the report structure is ready for drafting
Core logic unit test expansion	Hongshuo HU	EditMode suites for turn flow, deck logic, card resolution, grid logic, AI, and unit state	Core rules have direct automated coverage
Scene-coupled and integration testing	Zehan CHEN	UI/bootstrap tests plus a PlayMode smoke path for the battle loop	The main battle path has at least one end-to-end automated checkpoint
Metrics and evidence pack	Cheng YE	Coverage setup, execution statistics, matrices, charts, and bug-summary data	All report figures and tables have traceable raw inputs
Report draft	Guangjun HU	Complete draft with heavy table and chart emphasis	A full PDF draft exists
Final sync and scoring pass	All members, integrated by Guangjun HU	Final reruns, truth pass, formatting pass, and evidence consistency check	Report and evidence package are submission-ready

Table 1.4: Milestone structure used for the testing cycle.

Checkpoint area	Current owner	Responsibility in this report cycle	Main evidence source
Report integration	Guangjun HU	Maintain report structure, keep section language concise, integrate final evidence and appendix tables	‘doc/src/wr3-latex/**‘
Rule-focused EditMode suites	Hongshuo HU	Expand direct tests around stable game rules and manager classes	‘coursework/Assets/Tests/Editor/**
Scene and UI validation	Zehan CHEN	Cover bootstrap wiring, UI prompts, and smoke flows around the battle scene	‘coursework/Assets/Tests/Editor/** ‘course-work/Assets/Tests/PlayMode/**‘
Metrics and process evidence	Cheng YE	Prepare matrices, milestone records, package/tool evidence, and defect summaries	‘doc/src/project-records/testing/**‘, ‘doc/src/project-records/process/**‘

Table 1.5: Owner allocation during the testing phase.

Although this stage was a testing sprint, it was anchored to the broader implementation schedule already agreed in WR2. Reusing that timeline here makes

the sprint's place in the project sequence explicit without pretending that the testing cycle was planned in isolation.

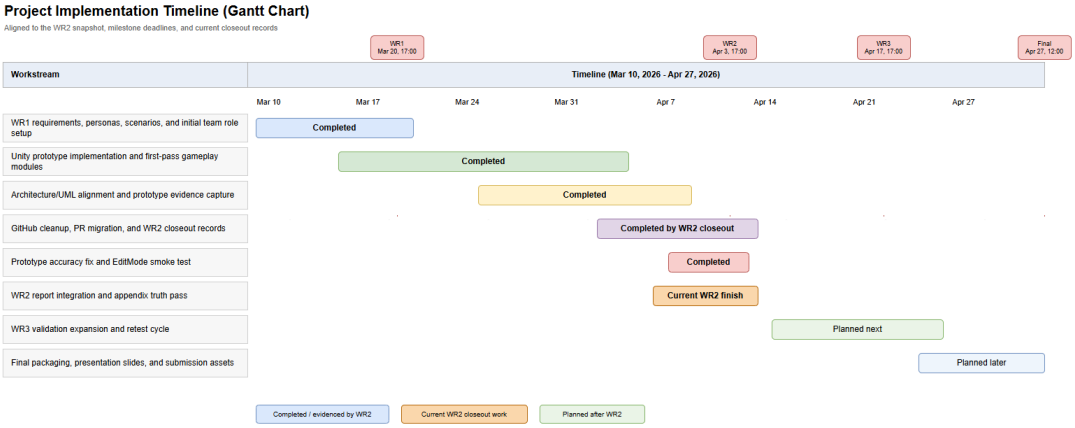

Figure 1.2: Project implementation timeline carried forward from WR2 to show where the testing sprint sits in the wider delivery sequence.

1.6 Workflow and Repository Control

The weekly sprint treated repository discipline as part of the evidence model. A test or document change was not counted as complete until it had both a stable place in the 'test' integration line and a corresponding rerun or archived evidence artifact.

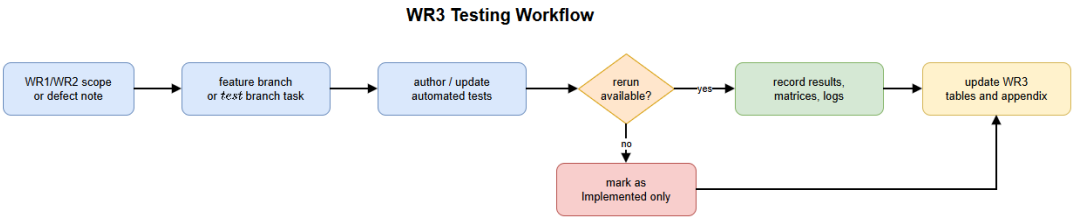

Figure 1.3: Testing workflow used to distinguish authored work from rerun evidence.

Control rule	Reason	Practical effect on the report cycle
Keep ‘test‘ as the integration branch	One branch makes execution, report updates, and validation records easier to align	The report can refer to one integration line while still noting milestone ownership.
Treat package/tool changes as report evidence	Coverage and testing tools are part of the validation setup, not invisible background work	‘manifest.json‘, ‘packages-lock.json‘, and package settings are included in the evidence set.
Do not promote discovery to verification automatically	Authored tests and visible tests are not the same as rerun-confirmed tests	The report uses ‘Verified‘, ‘Discovered‘, and ‘Implemented‘ labels consistently.
Freeze gameplay scope during the sprint	A weekly report is easier to evaluate when the validated slice is stable	Defect fixes and testability changes were allowed, but unrelated feature work stayed outside the evidence claims.

Table 1.6: Repository and evidence control rules used during the testing cycle.

Chapter 2

Test Strategy

This report uses a layered strategy because the project combines pure rule logic with scene-bound Unity behaviour. The final 2026-04-17 validation cycle confirms that the layered approach was appropriate: rule-dense managers were closed by consolidated EditMode reruns, while bootstrap and interaction flow were closed by scene-coupled EditMode tests plus one archived PlayMode smoke path [3, 1].

2.1 Verification and Validation Roles

Layer	**Question answered**	**Main methods**	**Expected evidence**
Verification	"Does the implemented code behave according to the intended rules?"	EditMode unit tests, focused component tests, bug-fix reruns	Pass/fail results, suite inventory, module coverage matrix
Validation	"Does the prototype still behave like the intended single-battle slice for a user?"	PlayMode smoke path, heuristic checks carried over from WR2, battle bootstrap tests	Smoke result, UI feedback checks, and verified player-facing fixes
Process validation	"Can the team show that testing work was planned, tracked, and reported honestly?"	Milestone tables, defect register, execution summary, appendix evidence index	Process chapter tables, synced supporting records, appendix source index

Table 2.1: Verification and validation responsibilities in this testing cycle.

2.2 TDD Use

The project followed a pragmatic TDD strategy rather than treating TDD as a slogan. For stable rule logic, the team used the normal *red-green-refactor* loop: write or extend a failing EditMode test first, make the smallest code change

needed to pass it, and then keep the test as part of the permanent rerun set. This was the main pattern for ‘TurnManager‘, ‘DeckManager‘, ‘CardResolver‘, ‘GridManager‘, ‘EnemyAI‘, and ‘UnitController‘.

For bug fixes, the team used *regression-first TDD*. Reported failures were first turned into executable checks, then fixed, and finally preserved as named rerun evidence. Scene-bound Unity classes required a lighter variant because not every behaviour can be isolated before object wiring exists, so ‘BattleUI‘, ‘GameRoot‘, ‘TileView‘, and ‘CardButtonView‘ were handled with focused component tests plus a final smoke rerun. This pattern was especially useful when closing the WR2 onboarding and enemy-phase feedback issues, because the fixes depended on small prompt and summary helpers rather than large gameplay rewrites.

TDD pattern	**Where it was used**	**How the team followed it**
Red-green-refactor	Rule-dense manager and controller logic	Add or extend a failing EditMode test first, implement the smallest passing change, then keep the case in the rerun suite.
Regression-first TDD	Known bugs and behavioural regressions	Convert a discovered failure into a named test before fixing it so the same issue cannot quietly return later.
Test-assisted iteration	Scene bootstrap, UI wiring, and interaction flow	Use focused EditMode component tests for deterministic parts, then confirm end-to-end wiring with the PlayMode smoke path.

Table 2.2: How TDD was applied in practice during the testing cycle.

2.3 Strategy Matrix

Test layer	Main targets	Reason for use	Validation use
EditMode unit tests	‘TurnManager‘, ‘DeckManager‘, ‘CardResolver‘, ‘GridManager‘, ‘EnemyAI‘, ‘UnitController‘	These classes are rule-dense and can be created with lightweight runtime objects in editor tests.	Primary evidence layer for core battle logic
Scene-coupled EditMode tests	‘BattleUI‘, ‘GameRoot‘, ‘TileView‘, ‘CardButtonView‘	These classes depend on Unity objects and UI/event wiring but do not require a full real-time session.	Focused evidence for bootstrap, prompt logic, UI wiring, and turn-summary feedback
PlayMode smoke test	Main battle start path	A small end-to-end checkpoint is needed so the report does not rely only on isolated rules.	One archived smoke suite for scene initialisation and first playable turn
Manual / heuristic follow-up	Onboarding clarity, enemy-turn readability	These are player-facing quality issues that were first identified heuristically before being translated into targeted UI and turn-summary checks.	Used to identify the two inherited WR2 issues and confirm the direction of the final fixes

Table 2.3: Test strategy matrix.

2.4 Environment and Tooling

Environment item	Recorded value	Why it matters
Unity editor version	‘6000.3.11f1‘	All test execution and report claims are scoped to this editor version.
Project branch	‘test‘ integration branch	Keeps the evidence aligned with one integration line rather than scattered local experiments.
Primary automation framework	‘com.unity.test-framework‘ ‘1.6.0‘	Supplies EditMode and PlayMode test execution.
Coverage tooling	‘com.unity.testtools.codecoverage‘ ‘1.3.0‘	Produces the archived quantitative evidence set, including ‘74.2%‘ line coverage and ‘92.4%‘ method coverage.
UI dependencies under test	‘com.unity.ugui‘ ‘2.0.0‘, ‘com.unity.inputsystem‘ ‘1.19.0‘	Important because UI and click handling are part of the tested battle slice.
Execution path recorded	Consolidated reruns + archived coverage export	The report cites the final EditMode rerun, the PlayMode smoke rerun, and a batchmode coverage archive rather than discovery-only editor visibility.

Table 2.4: Testing environment and toolchain used by the validation cycle.

2.5 Scope and Code-Change Guardrails

Guardrail	Allowed change	Disallowed change
Gameplay scope	Keep the single-battle vertical slice and its existing four-card ruleset	Add unrelated new gameplay systems just to inflate report content
Testability refactors	Small non-behavioural changes that clarify setup, state control, or dependency wiring for tests	Behaviour-changing refactors that rewrite the prototype under the name of testing
Evidence claims	Claim ‘Verified‘ only when a named rerun or archived export exists; keep implementation status separate from validation status	Re-use start-of-cycle ‘Implemented‘ or ‘Discovered‘ labels as if they still describe the final state
Coverage metrics	Archive raw OpenCover XML, generated reports, and summary tables before citing quantitative results	Fabricate or round-trip coverage numbers without a traceable export on disk

Table 2.5: Guardrails that keep the report focused on validation rather than feature drift.

2.6 Coverage Strategy

- The first quantitative layer is **suite volume**: the scoped suite contains ‘58‘ test methods across ‘11‘ suite files.
- The second quantitative layer is **traceability**: the report includes both a requirements coverage matrix and a module/class coverage matrix, and both now resolve to rerun-backed evidence.
- The third quantitative layer is **execution confirmation**: all ‘58‘ authored test methods are backed by named rerun records in the archived 2026-04-17 execution set.
- The fourth quantitative layer is **line and method coverage**: the archived report records ‘868/1169‘ covered lines (‘74.2%‘) and ‘146/158‘ covered methods (‘92.4%‘).
- The lowest-covered runtime target is still ‘BattleUI‘ at ‘56.2%‘ line coverage, but it is now directly exercised by ‘19‘ tests and is no longer treated as an unaddressed evidence gap.

Chapter 3

Test Plan

The test plan was built backward from the codebase rather than from generic test-type labels. In the final 2026-04-17 report state, the plan is no longer prospective: every scoped runtime class has a direct suite, the smoke path has been rerun, and the supporting records now describe a closed validation cycle rather than a future rerun intention.

3.1 Requirements Coverage Matrix

Table 3.1: Requirements coverage matrix for the final single-battle validation set.

Req. ID	Requirement / expected behaviour	Coverage evidence and validation note
R-01	The battle scene must initialise a single playable tactical battle.	Primary suites: `GameRootTests` and `BattleFlowPlayModeTests`. Status: Verified. Confirmed by direct bootstrap tests and the PlayMode smoke rerun.
R-02	Units must register to the correct team and occupy valid starting tiles.	Primary suites: `GameRootTests` and `GridManagerTests`. Status: Verified. Registration, placement, and occupancy checks all passed in the consolidated rerun.
R-03	The player turn must reset card plays and draw a fresh hand each round.	Primary suites: `TurnManagerTests` and `DeckManagerTests`. Status: Verified. Turn flow and deck-hand lifecycle are both backed by named reruns.
R-04	Move cards must allow valid tile movement only.	Primary suites: `CardResolverTests` and `GridManagerTests`. Status: Verified. Movement range, placement, and execution branches all passed.
R-05	Strike cards must damage adjacent enemies and reject friendly targets.	Primary suites: `CardResolverTests` and `BattleUITests`. Status: Verified. Direct resolver and UI-triggered strike behaviour are both rerun-backed.
R-06	Guard cards must apply a temporary defensive state.	Primary suites: `CardResolverTests`, `UnitControllerTests`, and `BattleUITests`. Status: Verified. Guard resolution, guard consumption, and UI guard flow all passed.

Req. ID	Requirement / expected behaviour	Coverage evidence and validation note
R-07	Push cards must damage the target and attempt knock-back.	Primary suites: `CardResolverTests` and `GridManagerTests`. Status: Verified. Damage plus movement constraints are covered by passing rerun evidence.
R-08	Enemy turns must target the nearest alive player and either move or attack.	Primary suites: `EnemyAITests` and `TurnManagerTests`. Status: Verified. AI selection logic and turn-loop orchestration both have passing rerun evidence.
R-09	Battle end must lock the loop and expose a consistent outcome state.	Primary suites: `TurnManagerTests` and `BattleUITests`. Status: Verified. Earlier regression evidence was preserved and the broader rerun passed.
R-10	The UI must prompt the player when selection or target conditions are invalid.	Primary suites: `BattleUITests` and `CardButtonViewTests`. Status: Verified. Direct UI prompt and button-binding suites both passed in the consolidated rerun.

3.2 Module and Class Coverage Summary

Table 3.2: Module/class coverage summary for the final validation set.

Runtime target and direct evidence	Covered behaviours and validation status

`TurnManager`
`TurnManagerTests`
3 tests
	Battle start without valid sides, card-play quota consumption, end-turn transition, and battle-end lock. Status: Verified.

`DeckManager`
`DeckManagerTests`
6 tests
	Fallback deck build, draw count, discard handling, reshuffle behaviour, and the empty-deck branch. Status: Verified.

`CardResolver`
`CardResolverTests`
6 tests
	Valid target ranges plus move, strike, guard, push, and invalid/friendly target rejection. Status: Verified.

`GridManager`
`GridManagerTests`
7 tests
	Grid generation, bounds, placement, movement, push limits, neighbour lookup, BFS range, and highlight reset. Status: Verified.

`EnemyAI`
`EnemyAITests`
4 tests
	Manhattan distance, nearest-target selection, step-direction priority, and dead-target skipping. Status: Verified.

`UnitController`
`UnitControllerTests`
6 tests
	HP init, damage, guard reduction, healing clamp, death deactivate, and position sync. Status: Verified.

`BattleUI`
`BattleUITests`
19 tests
	Card selection prompts, strengthened next-step messaging, enemy/action summaries, guard auto-resolve, invalid target rejection, action-log updates, and battle-end UI lock. Status: Verified.

`GameRoot`
`GameRootTests`
2 tests
	Dependency wiring, scene bootstrap, unit registration, and initial battle start. Status: Verified.

Runtime target and direct evidence	Covered behaviours and validation status

`TileView`
`TileViewTests`
3 tests
	Coordinate init, occupancy changes, and visual highlight method safety. Status: Verified.

`CardButtonView`
`CardButtonViewTests`
1 test
	UI button-to-card selection binding. Status: Verified.

Smoke path
`BattleFlow`
`PlayModeTests`
1 test
	Whole battle bootstrap and first playable state. Status: Verified.

3.3 Final Suite Inventory

Table 3.3: Final suite inventory used by the validation results.

Suite file, mode, and method count	Main role in the validated suite
`TurnManagerTests` EditMode, 3 methods	Preserve the battle-end regression layer and verify turn-loop correctness.
`DeckManagerTests` EditMode, 6 methods	Stabilise hand, deck, discard, and reshuffle rules.
`EnemyAITests` EditMode, 4 methods	Separate deterministic AI rules from turn-loop orchestration.
`UnitControllerTests` EditMode, 6 methods	Lock state changes around HP, guard, death, and movement.
`GridManagerTests` EditMode, 7 methods	Cover the grid as a shared dependency for move, strike, and push logic.
`CardResolverTests` EditMode, 6 methods	Test card semantics directly without UI indirection.
`BattleUITests` EditMode, 19 methods	Capture UI prompt logic, strengthened onboarding and enemy-phase summaries, auto-resolve behaviour, helper branches, and action-log updates.
`GameRootTests` EditMode, 2 methods	Verify scene bootstrap and dependency discovery without a full play session.
`TileViewTests` EditMode, 3 methods	Verify tile coordinate, occupancy, and highlight behaviour used by grid/UI flows.
`CardButtonViewTests` EditMode, 1 method	Verify the click-binding handoff from button to battle UI.

`BattleFlow`
`PlayModeTests`
PlayMode, 1 method
	Provide one end-to-end smoke checkpoint for battle bootstrap and first playable turn.

The test plan has therefore closed its original gaps: every targeted runtime class is directly tested, the smoke path is rerun-backed, and the supporting records now describe a validated suite rather than a hypothetical one.

Chapter 4

Validation Results

This chapter reports the final 2026-04-17 validation results rather than an intermediate checkpoint. All scoped suites now have named rerun records, the combined coverage export is archived on disk, and the inherited WR2 usability findings are now closed by the final rerun-backed evidence pack rather than carried as open blockers.

4.1 Execution Summary

Date	Mode	Scope	Total	Passed	Status
2026-04-17	EditMode	Consolidated rerun of all EditMode suites on the mirrored coverage workspace	57	57	Verified
2026-04-17	PlayMode	Coverage-backed smoke rerun for the battle start path	1	1	Verified
2026-04-17	Coverage export	Combined EditMode + PlayMode report generation and archive writeback	58	58	Archived

Table 4.1: Final execution summary used by the validation report.

Execution measure	Value	Interpretation
Total suite files	‘11‘	The scoped suite consists of ‘10‘ EditMode files plus ‘1‘ PlayMode smoke file.
Total test methods	‘58‘	The current suite is large enough to support both dense appendix tables and quantitative reporting.
Verified by named rerun	‘58‘	Every authored test method is now backed by an archived rerun record.
Directly targeted runtime classes	‘10‘	Every scoped runtime target in the battle slice has at least one direct suite file.
Smoke / integration paths	‘1‘	The battle bootstrap and first playable turn are covered by an end-to-end PlayMode checkpoint.

Table 4.2: Execution position after the final reruns.

4.2 Coverage and Traceability Results

Coverage metric	Value	Evidence meaning
Runtime classes in report	‘11‘	The archived coverage report spans all runtime classes cited in the validation chapter.
Covered lines	‘868‘	Taken from the archived multi-report coverage summary.
Coverable lines	‘1169‘	Used as the denominator for the line-coverage claim.
Line coverage	‘74.2%‘	The report now has archived quantitative coverage evidence rather than package-only tooling claims.
Covered methods	‘146‘	Shows that the suite closes most method-level behaviour in the scoped battle slice.
Total methods	‘158‘	Method totals come directly from the archived coverage summary.
Method coverage	‘92.4%‘	Method coverage is materially stronger than the line aggregate, which is expected for UI-heavy scripts.

Table 4.3: Archived coverage totals.

Figure 4.1: Class-level line coverage from the archived report. ‘BattleUI‘ remains the lowest-covered runtime class, but the final clean rerun lifted it to ‘56.2%‘, above the report target line and backed by direct tests plus the PlayMode smoke path.

Coverage evidence item	**Status**	**Reason it is report-ready**
Requirements coverage matrix	Archived	All ‘10/10‘ scoped requirements now map to rerun-backed suites.
Module/class coverage matrix	Archived	Every targeted runtime class now has a direct verified suite file.
Execution summary table	Archived	The final EditMode rerun, PlayMode smoke rerun, and coverage export are all named records on disk.
Line-coverage chart	Archived	The chart is derived from the archived 2026-04-17 coverage report rather than draft estimates.
Representative test case appendix	Archived	The appendix now references only final verified suite labels.

Table 4.4: Evidence readiness for the final report.

4.3 Defect Summary

ID	Area	Severity	Finding	Status
WR2-BUG-01	Onboarding feedback	Medium	New-player guidance around the required unit-before-card order was previously too easy to miss.	Fixed and verified
WR2-BUG-02	Enemy turn readability	Medium	Enemy actions were functionally correct, but the phase summary previously lacked enough emphasis.	Fixed and verified
WR2-BUG-03	Battle-end handling	High	The battle loop previously risked continuing after elimination.	Fixed and verified
WR3-BUG-01	Test execution pipeline	Medium	Early live-editor discovery exposed only part of the authored suite, creating a truth gap between inventory and rerun evidence.	Fixed and verified
WR3-BUG-02	Coverage evidence gap	Low	Code Coverage was installed before the first archived export existed.	Fixed and verified

Table 4.5: Final defect summary.

ID	Probable root cause	Fix or mitigation already applied	Prevention / next action
WR2-BUG-01	The prototype relied on weak onboarding affordances around the unit-before-card rule.	Prompt wording and next-step messaging were strengthened, and the final ‘BattleUI‘ rerun verified the new flow.	Keep the current prompt flow as a regression-backed behaviour and reretest after any wording or highlight change.
WR2-BUG-02	Enemy-phase feedback was functionally correct but visually understated.	Enemy summaries and related ‘BattleUI‘ / ‘TurnManager‘ message paths were strengthened and verified in the final rerun.	Rerun the summary/message checks after any later wording, log, or transition-layout change.
WR2-BUG-03	End-state logic was previously too weakly locked.	‘TurnManagerTests‘ verified the battle-end fix and the consolidated rerun confirmed it remained stable.	Keep this suite as a permanent regression gate in future reruns.
WR3-BUG-01	The editor refresh and discovery state initially did not match the full authored inventory.	Missing test metadata was corrected and the final rerun upgraded the suite to fully verified status.	Use named rerun records rather than discovery-only editor visibility for future truth claims.
WR3-BUG-02	The coverage tool was not part of the original prototype workflow.	The package was installed, batchmode coverage export was scripted, and the final archive was written under ‘coverage/2026-04-17/‘.	Refresh the same export path whenever WR3 evidence is extended later.

Table 4.6: Root cause, mitigation, and prevention view for the final defect register.

The validation result is therefore stable rather than provisional. The report now has a rerun-backed suite, an archived quantitative coverage layer, and a defect register in which the inherited WR2 issues and the tooling blockers are all closed with named evidence.

Chapter 5

Summary and Reflections

This work changes the report from "we should test more" to "the scoped suite has been rerun, archived, and cross-referenced consistently." That is a much stronger position than the project had at the start of the testing cycle, and it means the remaining discussion can focus on regression watchpoints and optional presentation polish rather than missing core evidence.

5.1 Outcome

Work item	**Result**	**Comment**
Report scaffold	Completed	Formal LaTeX source exists with the chapter structure, appendix, and compiled PDF.
Milestones and ownership plan	Completed	The report and process records now share the same M1–M6 structure and closure language.
Direct runtime test coverage	Verified	Every targeted runtime class now has a direct rerun-backed suite file.
Coverage tooling and export	Archived	Code Coverage package setup, raw XML, HTML report, badges, and summary files are archived.
Execution confirmation of the expanded suite	Completed	The scoped suite is now '58/58' verified by named rerun evidence.
Final quantitative evidence pack	Archived	Execution summary, matrices, defect register, milestone status, and coverage totals are all on disk.

Table 5.1: Outcome summary for the final validation cycle.

5.2 Milestone Closure

Milestone	Status	Closure evidence
M1. Scaffold and planning setup	Completed	The source tree, requirements structure, and suite inventory were prepared before the rerun cycle.
M2. Core logic unit test expansion	Completed	The full EditMode rerun passed ‘57/57‘, covering all scoped core logic targets.
M3. Scene-coupled and integration testing	Completed	UI/bootstrap suites and the PlayMode smoke path passed and are archived.
M4. Metrics and evidence pack	Completed	Coverage export, matrices, defect records, and supporting summaries are archived under ‘project-records‘.
M5. Report draft	Completed	The LaTeX draft has been promoted from checkpoint wording to rerun-backed validation wording.
M6. Final sync and scoring pass	Completed	The 2026-04-17 validation pass aligns the report, appendix, supporting records, and compiled PDF.

Table 5.2: Milestone closure view for the 2026-04-17 validation pass.

5.3 Optional Follow-up Backlog

Priority	Task	Status	Reason it remains optional
P1	Retest ‘BattleUI‘ prompt and summary flows after any later wording, log, or layout change.	Regression watch	The class has already crossed the target line at ‘56.2%‘; future work is mainly about preserving the verified behaviour.
P1	Keep the unit-before-card onboarding cues stable during any final text or highlight polish.	Regression watch	‘WR2-BUG-01‘ is already fixed and verified, so the remaining work is to avoid reintroducing the issue.
P1	Keep enemy-turn summary emphasis stable during any final UI presentation polish.	Regression watch	‘WR2-BUG-02‘ is already fixed and verified, so the remaining work is presentation consistency rather than defect closure.
P2	Add stronger branch / PR / tag screenshots if a richer appendix is desired.	Optional	This would strengthen process presentation, but it does not change the evidence already archived.

Table 5.3: Optional backlog after the validation cycle.

5.4 Reflection

Three points matter most at the end of the validation cycle.

- The project is no longer short of tests in an absolute sense. The current suite inventory is large enough to support a substantial appendix and a data-driven report body.
- The main engineering gain is not just higher test count; it is the conversion of test count into traceable rerun evidence, archived coverage output, and defect-status discipline.
- The remaining risk has moved out of execution evidence and into optional presentation polish plus regression watch on already-fixed UI messaging. That is a better risk profile for submission because it is visible, bounded, and honestly reported.

The practical lesson is therefore simple: once the suite and coverage export exist, the highest-value work is a strict truth pass. Reopening scope would weaken the report more than it would help it unless the new work were rerun and re-documented immediately.

Reference

[1] Group 2. Wr3 preparation test and bug note, 2026. Local preparation log stored at doc/src/project-records/testing/wr3-prep-log-2026-04-01.md.

[2] Group 2. Wr3 sample scoring summary, 2026. Local summary extracted from the high-scoring sample and stored at doc/src/wr3-latex/wr3-scoring-summary.md.

[3] University of Nottingham. Comp4135 coursework specification, 2026. Local course specification used for WR3 structure, process evidence, and testing expectations.

[4] University of Nottingham. Comp4135 weekly report 3 template, 2026. Template PDF stored under doc/src/wr3-latex.

Appendix A

Supporting Evidence Index

Source file	Type	Use in this report
‘doc/src/project-records/testing/wr3-test-inventory-2026-04-17.md‘	Testing record	Source for suite counts, module coverage, and appendix planning
‘doc/src/project-records/testing/wr3-execution-summary-2026-04-17.md‘	Testing record	Source for the final rerun-backed execution summary and suite totals
‘doc/src/project-records/testing/wr3-requirements-coverage-matrix-2026-04-17.md‘	Testing record	Source for Table 3.1
‘doc/src/project-records/testing/wr3-coverage-export-2026-04-17.md‘	Testing record	Source for the archived coverage totals, class metrics, and export-path narrative
‘doc/src/project-records/testing/wr3-defect-log-2026-04-17.md‘	Testing record	Source for the defect tables in Chapter 4
‘doc/src/project-records/process/wr3-milestone-status-2026-04-17.md‘	Process record	Source for the final milestone closure table in Chapter 5
‘doc/src/project-records/testing/coverage/2026-04-17/Report/‘	Coverage archive	Contains the HTML coverage report, badges, and summary files cited in Chapter 4
‘doc/src/project-records/testing/prototype-accuracy-editmode-run-2026-04-02.md‘	Testing record	Earlier regression record for the battle-end fix referenced in Chapter 4
‘doc/src/project-records/process/wr3-execution-plan-2026-04-17.md‘	Process record	Canonical WR3 scope, milestones, testing targets, and acceptance gates
‘doc/src/project-records/process/wr3-milestones-and-owners-2026-04-17.md‘	Process record	Source for the milestone, owner, and completion-evidence tables
‘doc/src/project-records/process/README.md‘	Process record	Explains how repo and board evidence packages are organised
‘doc/src/wr3-latex/wr3-scoring-summary.md‘	Local sample analysis	Drives the report’s table-heavy structure and

A.1 Testing and Validation Meeting

Date: 15/04/2026
Format: Online
Chair: Guangjun HU
Secretary: Zehan CHEN

A.1.1 Attendance

- Guangjun HU – present
- Hongshuo HU – present
- Cheng YE – present
- Zehan CHEN – present

A.1.2 Agenda

1. Review WR2 heuristic walkthrough findings and defect triage.
2. Confirm the test strategy and automated test scope.
3. Allocate report drafting tasks.
4. Set deadlines for remaining open bugs and final submission preparation.

A.1.3 Progress Since Last Meeting

Since the WR2 closeout meeting, the team has:

- Confirmed the WR2 closeout state on 02/04/2026 before starting the validation work.
- Applied the battle-end accuracy fix and verified it with EditMode tests.
- Created the initial test log and defect log under `doc/src/project-records/testing/`.

A.1.4 Key Discussion Points

- The high-severity battle-end gap (`WR2-BUG-03`) was confirmed fixed by the EditMode smoke test. No additional runtime changes were required for this issue.
- Onboarding clarity (`WR2-BUG-01`) and enemy-phase emphasis (`WR2-BUG-02`) were judged medium severity at triage time, then carried into the final polish pass and closed with rerun-backed evidence before the evidence freeze.
- The team agreed to keep the report within the course template structure and not to introduce new gameplay features during the validation cycle.
- GitHub pull-request review remains the canonical collaboration path; final submission changes must pass through reviewed and merged pull requests.

A.1.5 Decisions Made

- Scope is limited to testing documentation, defect tracking, and report assembly.
- All four members will contribute to the final report draft, with Cheng YE leading LaTeX compilation and Hongshuo HU leading runtime verification.
- The two inherited medium-severity usability issues were accepted into the final polish pass and had to be closed with named rerun evidence before the evidence freeze.

A.1.6 Action Items

Task	**Owner**	**Due**	**Review owner**
Draft LaTeX sections and tables	Cheng YE	16/04/2026	Guangjun HU
Run EditMode tests and capture results	Hongshuo HU	15/04/2026	Guangjun HU
Update defect register and meeting minutes	Zehan CHEN	16/04/2026	Cheng YE
Final integration check in Unity editor	Hongshuo HU	17/04/2026	Whole team

Table A.2: Action items agreed in the testing and validation meeting.

A.1.7 Issue Closure Status

- `WR2-BUG-01`: fixed and verified by the final `BattleUI` rerun after the prompt and next-step messaging updates.
- `WR2-BUG-02`: fixed and verified by the final `BattleUI` and `TurnManager` reruns after the enemy summary emphasis updates.
- Automated coverage for `GridManager` and `CardResolver` was pending at the time of the meeting and was later completed during the final coverage pass.

A.1.8 Next Meeting

- Tentative date: 24/04/2026.
- Main agenda: final submission checklist, closed-bug retest review, and repository submission-readiness confirmation.

Appendix B

Representative Test Case Inventory

Table B.1: Representative test case inventory for the final suite.

Module / suite	**Test case**	**Purpose**	**Label**
TurnManager	`StartBattle_WhenNoEnemyUnits_EndsInPlayerVictory`	Verify immediate victory when the enemy side is empty at battle start.	Verified
TurnManager	`StartBattle_WhenNoPlayerUnits_EndsInEnemyVictory`	Verify immediate defeat when the player side is empty at battle start.	Verified
TurnManager	`EndPlayerTurn_WhenEnemyDefeatsLastPlayer_DoesNotRestartPlayerTurn`	Prevent the loop from incorrectly starting a new player turn after defeat.	Verified
DeckManager	`ResetDeck_WithoutStartingDeck_UsesFallbackDeck`	Confirm the fallback deck is created when no starting deck is configured.	Verified
DeckManager	`DrawForTurn_DrawsConfiguredNumberOfCards`	Confirm the turn draw count uses the configured cards-per-turn value.	Verified

Module / suite	Test case	Purpose	Label
DeckManager	`DiscardFromHand_MovesCardToDiscardPile`	Confirm single-card discard behaviour.	Verified
DeckManager	`DiscardHand_MovesAllCardsToDiscardPile`	Confirm whole-hand discard behaviour at turn reset.	Verified
DeckManager	`Draw_WhenDrawPileEmpty_ReshufflesDiscardPile`	Confirm discard-to-draw reshuffle behaviour.	Verified
DeckManager	`Draw_WhenNoCardsRemain_LeavesHandUnchanged`	Confirm the empty-deck branch exits safely.	Verified
EnemyAI	`ManhattanDistance_ReturnsAbsoluteGridDistance`	Validate the AI distance metric used for targeting.	Verified
EnemyAI	`FindNearestPlayer_SkipsDeadUnitsAndReturnsClosestAliveTarget`	Confirm the AI ignores dead targets and chooses the nearest alive player.	Verified
EnemyAI	`GetStepToward_PrefersHorizontalStepWhenAxesTie`	Confirm deterministic tie-breaking in step selection.	Verified
EnemyAI	`GetStepToward_UsesVerticalStepWhenVerticalDeltaIsLarger`	Confirm vertical movement preference when vertical distance is larger.	Verified
UnitController	`CreateUnit_InitializesCurrentHpFromConfiguredValue`	Confirm unit creation respects the configured max HP.	Verified
UnitController	`TakeDamage_ReducesHitPoints`	Confirm standard damage application.	Verified

Module / suite	Test case	Purpose	Label
UnitController	`Guarding_ReducesNextHitAndThenClearsGuard`	Confirm one-hit guard consumption behaviour.	Verified
UnitController	`Heal_DoesNotExceedMaxHp`	Confirm healing clamps at max HP.	Verified
UnitController	`TakeDamage_ToZero_DisablesGameObject`	Confirm unit death deactivates the object.	Verified
UnitController	`SetGridPosition_UpdatesGridAndWorldPosition`	Confirm grid/world position synchronisation.	Verified
GridManager	`GenerateGrid_CreatesExpectedTiles`	Confirm grid size and bounds setup.	Verified
GridManager	`TryPlaceUnit_PlacesUnitAndUpdatesOccupancy`	Confirm unit placement and occupancy bookkeeping.	Verified
GridManager	`TryMoveUnit_MovesUnitAndClearsPreviousTile`	Confirm movement updates both origin and destination tiles.	Verified
GridManager	`TryPushUnit_RejectsOutOfBoundsDestination`	Confirm push logic rejects invalid destinations.	Verified
GridManager	`GetNeighbors_ReturnsOrthogonalInBoundsCoordinates`	Confirm neighbour enumeration.	Verified
GridManager	`GetTilesInRange_ReturnsBreadthFirstManhattanRange`	Confirm BFS range expansion used by move/attack highlighting.	Verified
GridManager	`HighlightMoveRange_TracksHighlightedTilesAndClearHighlightsResetsState`	Confirm highlights_ tracking and cleanup.	Verified
CardResolver	`GetValidTiles_Move_ReturnsUnoccupiedTilesWithinRange`	Confirm move-card targeting excludes occupied tiles.	Verified

Module / suite	Test case	Purpose	Label
CardResolver	`ResolveTileCard_Move_MovesUnitToTargetTile`	Confirm move-card execution updates the unit and grid.	Verified
CardResolver	`ResolveTileCard_Guard_SetsUnitGuarding`	Confirm guard-card resolution sets the defensive state.	Verified
CardResolver	`ResolveUnitCard_Strike_DamagesEnemyAndClearsTileWhenTargetDies`	Confirm strike damage and occupancy cleanup on death.	Verified
CardResolver	`ResolveUnitCard_Push_DamagesAndMovesEnemyBack`	Confirm push damage plus displacement.	Verified
CardResolver	`ResolveUnitCard_RejectsFriendlyTarget`	Confirm enemy-only targeting rules.	Verified
BattleUI	`SelectCard_WithoutSelectedUnit_RequestsUnitSelection`	Confirm the UI prompts for unit selection before card use.	Verified
BattleUI	`SelectCard_GuardCardWithSelectedUnit_AutoResolvesAndConsumesCardPlay`	Confirm self-target guard cards resolve immediately.	Verified
BattleUI	`TryPlayCardOnTile_InvalidTarget_ReturnsFalseAndKeepsCardSelected`	Confirm invalid tile targets do not resolve and keep state visible.	Verified
BattleUI	`TryPlayCardOnUnit_ValidStrike_ConsumesPlayAndUpdatesActionLog`	Confirm valid strike actions update both play count and action log.	Verified
BattleUI	`SelectCard_AfterBattleEnded_ShowsBattleOutcome`	Confirm UI lock and outcome message after battle end.	Verified
GameRoot	`Awake_FindsDependenciesWhenSerializedFieldsAreEmpty`	Confirm dependency discovery in the scene bootstrap object.	Verified

Module / suite	Test case	Purpose	Label
GameRoot	`Start_InitializesManagersPlacesUnitsAndStartsBattle`	Confirm bootstrap wiring from managers to first battle state.	Verified
TileView	`Initialize_SetsCoordinate`	Confirm tile coordinate initialisation.	Verified
TileView	`SetOccupantAndClearOccupant_UpdateOccupancyState`	Confirm tile occupancy bookkeeping.	Verified
TileView	`HighlightMethods_CreateRuntimeMaterialAndDoNotThrow`	Confirm tile highlight methods are safe to call at runtime.	Verified
CardButtonView	`Bind_ClickInvokesBattleUiSelection`	Confirm button binding forwards the selected card to the battle UI.	Verified
PlayMode smoke	`GameRoot_Start_InitializesBattleReadyState`	Confirm the scene reaches a playable battle-ready state in PlayMode.	Verified